

Interview Notes

14 videos · ek jagah · concept-clear · Hinglish

Repeated content ek baar, code snapshots hata diye gaye, har naye video ka alag rang.

Index — video pe click karo

01	LLM Evals Kyun Zaroori Hain? Vibe testing ki problem, 3 real-world failures, software testing vs LLM evaluation, playlist roadmap	p. 2
02	LLM Evals Kya Hote Hain? Formal definition, 3 characteristics, Model Evals vs Application Evals	p. 4
03	LLM Application Eval Workflow 9-step universal workflow + Zomato email classifier example	p. 6
04	Ek App, Multiple Eval Pipelines Kyun? Multiple failure points aur multiple risk categories	p. 8
05	Eval Methods: Programmatic, Human, LLM-as-Judge Kaun evaluate karega + reference-based vs reference-free	p. 10
06	Offline vs Online Evaluation Deploy se pehle aur baad ki eval, signals, logging, self-improving loop	p. 13
07	Model Evals — Why & What AI Engineer ko model evals kyun chahiye, benchmarks vs custom evals	p. 16
08	LLM Ki 8 Core Capabilities Har benchmark inhi me se kisi ek ko target karta hai	p. 18
09	LLM Benchmarks Anatomy, eval harness, limitations, knowledge benchmarks ki evolution (MMLU se HLE)	p. 22
10	LLM Leaderboards 4 types, kyun blindly trust nahi, AI Engineer ki tarah kaise padhein	p. 28
11	Custom Model Evals — Text-to-SQL Case Study Requirements, shortlisting, custom eval (Cricinfo example)	p. 31
12	RAG Evaluation — Strategy Overview Eval Suite: 3 levels + regression + online + interview answer	p. 36
13	RAG Retriever Evaluation Recall, Precision, golden dataset, DeepEval, live improvements	p. 41
14	Generator, RAG Triad & Application Metrics Faithfulness, Answer Relevancy, Contextual Relevancy, GEval	p. 46

Callout boxes ka matlab

Note

Extra detail

Key Insight

Concept ki chaabi

Interview Tip

Interview me bolne layak

Yaad Rakho

Pehle dekha hua, sirf recall

Source Note

Source me mismatch/uncertainty

Danger

Real-world nuksaan

LLM Evals Kyun Zaroori Hain?

Vibe testing ki problem, 3 real-world failures, software testing vs LLM evaluation, playlist roadmap

IS VIDEO ME Vibe testing · Air Canada / Chevrolet / Colombian airline · Software testing vs LLM eval · Playlist roadmap

1 LLM Evals Kya Hai — Aur Vibe Testing Kyun Galat Hai

LLM Evals = LLM-based application ko deploy karne se pehle **properly test/evaluate** karne ka process. Sirf 4-5 questions poochh kar "lag raha hai sahi kaam kar raha hai" bol dena eval nahi hai.

Vibe Testing — sabse common (galat) approach

- **Definition:** Casually kuch prompts try karke, sirf feel ke basis pe judge karna ki app sahi kaam kar rahi hai ya nahi — koi metric nahi lagaya jaata.
- **Example:** 5-10 questions poochhe, answers achhe lage, so maan liya "it works".

Vibe Testing ki 4 problems

Problem	Matlab
Informal	Koi structured process nahi hota.
Subjective	Person to person judgement alag hoga.
Not repeatable	Next version banate waqt same tarike se dobara evaluate nahi kar paoge.
Sirf chhote projects ke liye	Personal/portfolio projects me chal jaata hai; production-grade system ke liye nahi.

2 Real-World Case Studies — Bina Eval Deploy Karne Ke Nateeje

(a) Air Canada — Bereavement Fare Hallucination

- Customer ne chatbot se bereavement (death-in-family) discount policy poochhi.
- Chatbot ne **hallucinate** karke galat bataya: pehle full fare pay karo, baad me refund milega. Actual policy: discount pehle hi lena hota hai, baad me refund nahi milta.
- Customer court gaya. Air Canada ka argument: chatbot ek "separate entity" hai.
- **Verdict:** Website ki tarah chatbot bhi company ki property hai — jo bhi bole, company ownership legi. Air Canada haara aur poora refund dena pada.

(b) Chevrolet Dealer Chatbot — Jailbreak → \$1 Car Offer

- User ne chatbot ko convince/**jailbreak** kiya ki uski har baat maani jaaye.
- Phir poochha "\$1 me car doge?" — jailbroken chatbot ne agree karke likhit me binding offer de di.
- Poori conversation documented thi; screenshots social media pe viral hue → dealership/Chevrolet ki badnaami (negative marketing).

(c) Colombian Airline Case — Fabricated Legal Citations

- Injured passenger ke lawyer ne ChatGPT se past similar court cases maange (proof ke liye).
- Model ne confidently **fake cases fabricate** kar diye — fake names, dates, sab kuch.
- Lawyer ne bina verify kiye court me present kar diya; opposition ne check kiya to cases exist hi nahi karte the.
- Result: lawyer/firm pe ~\$5000 fine, aur case bhi haar gaye.

Common Lesson Teeno cases me developers ne LLM app banaya, sirf vibe-test kiya, aur seedha deploy kar diya. Proper evaluation hoti to ye incidents avoid ho sakte the.

3 Software Testing vs LLM App Evaluation

Aspect	Traditional software testing	LLM app evaluation
Output nature	Deterministic — same input, hamesha same output (calculator: $2+2 = 4$).	Probabilistic (LLM ke nature ki wajah se) — same input pe alag output aa sakta hai. E.g. "overfitting kya hai" — har baar thoda alag phrasing, koi galat nahi.
Judging criteria	Sirf ek benchmark: correctness (sahi ya galat).	Multi-dimensional : Factuality, Completeness, Tonality, Groundedness, Latency, Cost (per answer).

Note Ye dimensions application-specific hote hain — alag app ke liye alag cheezein important hongi.

4 Playlist / Learning Roadmap

1. LLM Evals kya hote hain — concept ek example se.
2. LLM Evals ka poora landscape — techniques aur tools ka high-level overview.
3. Evaluating LLMs — benchmark categories (model-level evaluation).
4. Evaluating LLM Applications — app-level evaluation.
5. Apna khud ka Eval Pipeline — golden dataset curate karna + rubrics define karna.
6. RAG-specific Evals.
7. Agent-based Evals.
8. Safety-based Evals.
9. Operational Evals — deploy ke baad bhi evaluate karte rehna (latency, tokens/sec, time-to-first-token, system load).

Interview Tip GenAI/AI Engineer interviews me commonly poochha jaata hai: "How do you evaluate your RAG application?" aur "How do you evaluate your agentic AI application?" Dono ke liye multi-dimensional eval, golden dataset aur RAG/agent-specific evals clear hone chahiye.

LLM Evals Kya Hote Hain?

Formal definition, 3 characteristics, Model Evals vs Application Evals

IS VIDEO ME Formal definition · 3 characteristics · Eval ≠ sirf metric · Model Evals vs Application Evals

1 LLM Evals — Formal Definition

Definition LLM Evals are systematic, repeatable tests used to judge an LLM or an LLM-powered system against clear criteria.

Characteristic	Matlab
Systematic	Vibe testing nahi. Proper dataset banta hai jisme har tarah ka edge case cover ho — e.g. real 100 user-chats uthake test dataset banana.
Repeatable	Prompt, model, retriever, chunking strategy — kuch bhi change karo, same dataset pe wahi tarike se dobara evaluate kar sako. Isi se Version 1 vs Version 2 ki comparison hoti hai.
Clear Criteria	Kis basis pe judge kar rahe ho, wo pehle se define ho (answer correct ho, simple ho, sirf course content se aaye, safe ho). Bina criteria = vibe testing.

2 Common Misconception — Eval Sirf Ek Metric Nahi Hai

Classic ML/DL background se aane walon ko lagta hai eval matlab accuracy/precision/recall jaisa ek metric. Ye galat hai — **LLM Eval = poor testing setup**, sirf ek number nahi. Isme shaamil hota hai:

- **Kya** test kar rahe ho (e.g. RAG ka retriever component).
- **Kis criteria** pe (e.g. retriever ka accuracy).
- Test me kaunsa **dataset** use hua.
- **Kab** — offline ya deploy hone ke baad (production me).
- Kaunse **tools** (e.g. RAG app ke liye Ragas).

3 Eval Kin Sawalon Ka Jawab Deta Hai?

- Kya model is particular task/application ke liye use ho sakta hai?
- Kya ye system ship karne layak (production-ready) hai?
- Prompt v2 ne Prompt v1 se improve kiya?
- Kya RAG answer retrieved context me grounded hai?
- Kya agent task sahi complete kar raha hai?
- Kya chatbot real users ke liye safe hai?
- Kya latency control me hai?

Note Eval ka goal ek score dena nahi hai — goal in practical questions ka answer dena hai.

4 Do Types Ke LLM Evals

Note "Model Evals" aur "Application Evals" official industry terms nahi hain — samjhane ke liye banaye gaye hain. Industry me dono ko combined "LLM Evals" hi bolte hain.

(A) Model Evals — LLM khud ko evaluate karna

- **Goal:** naya LLM release hote waqt uski capabilities test/benchmark karna.
- **8 core capabilities** (detail Video 8 me): Reasoning, Knowledge, Basic Math, Coding, Instruction Following, Long Context, Multimodal, Tool Use.
- Ye kaam usually **frontier labs** (OpenAI, Anthropic...) karte hain — AI Engineer ke roz ke kaam ka part nahi. Aapko sirf benchmarks **padhna** aana chahiye (literacy), taaki project ke liye sahi LLM choose kar sako.

Capability	Example benchmark
Knowledge + reasoning (science, history, law, medicine)	MMLU
Basic / grade-school math	GSM8K
Coding	SWE-Bench, HumanEval
Instruction following	IFEval
Long context handling	Needle in a Haystack
Multimodal	MMMU

(B) Application Evals — poore LLM-based application ko evaluate karna

- LLM app ka sirf ek component ("brain") hai. Ek proper app me aur bhi bahut kuch hota hai: User Interface, System Prompt, Tools/APIs, Orchestration code (LangGraph jaisa flow control, branching, parallel execution), application-level Guardrails, Output Parsers, Memory & Context, Retrieval system (embedding model, vector DB), aur deploy ke baad Monitoring & Feedback loop.
- **Smartphone analogy:** Snapdragon/MediaTek apne processor ka benchmark score publish karte hain, par sirf strong processor se poora phone achha nahi banta — camera, OS, battery, screen sab achha hona chahiye. Waise hi sirf strong LLM se poori app achhi nahi banti.
- **Do levels:** Component-level (retriever accurate? embedding model sahi? reranker sahi?) aur System-level (overall response quality, latency, cost-per-token).
- **Question style badal jaata hai:** "Can the model do this?" nahi, balki "Kya hamara product sahi se kaam karega?"
- **Example (CampusX chatbot):** Student ka question sahi answer hua? Course material use hua? Answer faithful/grounded tha? Beginner ke liye easy tha? Hallucination hua? Response fast tha? Chatbot safe hai?

	Model Evals	Application Evals
Kya test hota hai	LLM ki capabilities	Poora app / uske components
Kaun karta hai	Frontier labs, third-party evaluators	AI Engineers / product teams
Question	Can the model do this?	Will our product work?

Interview Tip Online/YouTube pe jab bhi koi "LLM Evaluation" padhata hai, ~99% cases me wo Application Evals hi hota hai — kyunki AI Engineer ka core kaam applications banana aur evaluate karna hai, models banake benchmark karna nahi.

LLM Application Eval Workflow

9-step universal workflow + Zomato email classifier example

IS VIDEO ME 9-step workflow · Zomato example · Method choose karna · Continuous loop

1 Universal Workflow — 9 Steps

Ye workflow simple LLM app ho ya complex agent/RAG system — dono ke liye same rehta hai.

- 1. Define Task & Target** — kya evaluate karna hai (e.g. poora system/workflow).
- 2. Define Success Criteria** — kis metric se pata chalega system sahi hai (classification → Accuracy).
- 3. Build Golden Dataset** — real past data (chats/emails) + manually correct labels; typically 50-500 rows.
- 4. Define Evaluation Method** — kaun evaluate karega: Automated (code), Human, ya LLM-as-judge.
- 5. Run the Model** — dataset system me bhejo, har input ka output generate hoga.
- 6. Evaluate the Results** — chosen method se score (e.g. Accuracy = correct / total).
- 7. Analyze the Results** — errors ka pattern samjho: galti prompt me hai, model me, ya data me?
- 8. Improve & Iterate** — prompt tweak / model change → dobara Step 5 se, jab tak satisfied na ho.
- 9. Deploy → Monitor → Feed Back** — production ke failures wapas golden dataset me add karo, loop dobara chalao.

2 Example Walkthrough — Zomato Email Classifier

Example Zomato ko roz bahut saare customer emails aate hain. Ek LLM system banana hai jo email padhke usko Billing / Technical / General me classify kare aur sahi team ko auto-route kar de. Prompt: "You are this customer agent that reads the email and decides where to route it." (5-10 min me ban jaata hai — par seedha deploy nahi karna!)

Workflow step	Zomato me kya hua
Task / Target	Poora classification system evaluate karna hai.
Success metric	Accuracy — 100 emails me se kitne sahi route hue (90/100 = 90%).
Dataset	Past emails manually labelled: "My card was charged twice" → Billing; "App crashes on login" → Technical; "What are your hours?" → General.
Eval method	Automated (simple Python script) — output fixed label hai, seedha compare ho sakta hai. Human/LLM judge unnecessary cost hota.
Run + Evaluate	Accuracy = 80% (80/100 sahi).
Analyze	20% errors kyun? (a) System prompt confusing — Billing/Technical me confuse. (b) Model weak/small, task samajh nahi pa raha.
Improve + Iterate	Prompt tweak → 90%. Manager: aur improve karo → bada/better LLM → 95%. Manager satisfied.
Deploy + Monitor	Test data pe 95% tha, par naya real customer data aane pe galtiyan ho sakti hain → continuous monitoring.
Feedback loop	Galat classify hua email (Billing ko Technical bola) → Golden Dataset me add → eval-improve loop dobara. Dataset gradually richer hota jaata hai.

3 Takeaways

Evaluation method kaise choose karein (detail Video 5 me)

Method	Kab use karein
Automated (code)	Output structured/fixed ho (labels, numbers) — fast & free. E.g. classification accuracy.
Human	Nuanced judgement chahiye (long text compare, semantic meaning) — accurate par costly & slow.
LLM-as-judge	Beech ka option: long/unstructured text (chatbot answers) automatically judge karna — human se sasta, scalable.

Key Insight Eval sirf pre-deployment step nahi hai. define → run → evaluate → analyze → improve → deploy → monitor → failures feed back — ye loop system ke deployment me rehne tak chalta rehta hai. Yehi **repeatability** hai: same golden dataset pe baar-baar prompt/model badalke versions compare karna.

Interview Tip "Single app = single eval" mat sochna. Real systems me component-level aur system-level, dono tarah ke multiple evals saath chalte hain (retriever eval, embedding eval, end-to-end RAG eval, latency eval...). Iska reason agle video me hai.

Ek App, Multiple Eval Pipelines Kyun?

Multiple failure points aur multiple risk categories

IS VIDEO ME Multiple failure points (RAG example) · Multiple risk categories · App-type wise breakdown

1 Key Idea — Ek LLM App, Kai Independent Eval Pipelines

Ek single eval pipeline kaafi nahi hoti. Iske peeche 2 reasons hain: **(1) Multiple Failure Points** aur **(2) Multiple Risk Categories**.

2 Reason 1 — Multiple Failure Points (RAG Example)

RAG setup: **Query** → **Retriever** (vector DB se relevant docs fetch) → **Generator** (query + retrieved context se answer banata hai). Failure 3 levels pe ho sakta hai:

Level	Kya fail ho sakta hai	Kaunsi eval chahiye
Component	Retriever galat docs fetch kare; ya Generator sahi context milne ke baad bhi hallucinate kare.	Retriever eval → context relevance . Generator eval → faithfulness/groundedness (answer sirf given context se bana?).
Workflow	Dono components individually theek hon, phir bhi combination fail ho.	Retriever + Generator ke combination ki alag eval.
Application	Component aur workflow sahi hon, phir bhi poora system fail — e.g. answer correct par 10 second lag rahe (latency).	Latency / cost jaisi application-level eval.

Workflow-level failure — example

- K=5 (top-5 docs). Sahi answer wala doc **D5** (lowest priority) me mila. Retriever ne apna kaam kar diya.
- Generator ko bola gaya tha "higher-priority docs ko zyada weight do" — usne D1-D4 (galat/unrelated) ke basis pe answer bana diya → galat answer.
- Dono component individually theek the, par combination fail hua. **Fix:** reranker add karo jo D5 ko top pe le aaye.

Key Insight Component-level evals pass hone ka matlab workflow sahi hone ki guarantee nahi. Workflow sahi hone ka matlab poore application ke sahi hone ki guarantee nahi. Har level pe alag eval chahiye.

3 Reason 2 — Multiple Risk Categories

Har failure point ke andar bhi alag-alag risks matter karte hain. Inhe 3 broad categories me organize karte hain:

Category	Sawal
Application Quality	Kya app apna actual job achhe se kar raha hai? (correct, relevant, complete answers)
Safety	Kya output harmful nahi hai? (toxic/dangerous/biased content, private data leak, jailbreak)
Operations	Kya system deploy hone par fast, cheap aur reliable chalta hai?

4 Risk Categories — App Type Wise Breakdown

Category / App type	Kya check hota hai
Quality — General LLM app (e.g. summarizer)	Correctness & Accuracy, Relevance, Completeness, Instruction Following (format/length follow hua ya nahi).
Quality — RAG	Context Relevance, Retriever Recall, Groundedness/Faithfulness, Citation Accuracy (source se link kar pa raha hai, jaise ChatGPT karta hai).
Quality — Agent	Tool Selection, Parameter Correctness, Task Completion, Error Recovery.
Quality — Multi-turn chatbot	Context Retention (purani conversation yaad), Clarification Behavior (ambiguous query pe sahi clarify).
Safety	Toxicity, Harmful Content (self-harm, weapons, illegal), Bias (sabko equally treat), Private Data Leak, Prompt Injection & Jailbreak Resistance.
Operations	Latency, Cost per Request, Token Efficiency, Error/Failure Rate, Latency under Load.

Interview One-Liner ~99.99% cases me ek LLM app me ek se zyada eval pipelines lagti hain — kyunki multiple failure points hote hain (component / workflow / application) aur har jagah multiple risk categories (quality / safety / operations) cover karni padti hain. E.g. ek pipeline latency ke liye, ek safety ke liye, ek correctness ke liye.

Eval Methods: Programmatic, Human, LLM-as-Judge

Kaun evaluate karega + reference-based vs reference-free

IS VIDEO ME Programmatic (Recall@K) · Human (5 types) · LLM-as-Judge (UPSC, MAE) · Reference-based vs free

1 Eval Method Kya Hai — 3 Types

Eval Method = wo mechanism jo decide karta hai LLM ka output good hai ya nahi. Simple sawal: **eval pipeline ko execute kaun kar raha hai?**

Method	Kaun execute karta hai
Programmatic / Deterministic	Ek program / code.
Human	Ek insaan judge karke score deta hai.
Model-graded / LLM-as-judge	Ek LLM khud judge banke evaluate karta hai.

Note Golden dataset banana alag activity hai (usually human hi banata hai). Yahan discuss ho raha hai ki final scoring kaun execute kar raha hai.

2 Method 1 — Programmatic (Example: Retriever Eval, Recall@K)

Setup: CampusX RAG chatbot ke **retriever component** ko evaluate karna hai.

- **Metric — Recall@K:** "Sabhi correct/relevant docs me se kitne system ne apne top-K results me retrieve kiye?"
Example: 2 correct docs (D1001, D1003), K=5 me sirf D1001 mila → Recall = $1/2 = 50\%$.
- **Golden dataset:** 50–100 questions; human expert ne har question ke liye vector DB me sahi document IDs manually identify kiye.
- **Method:** Python script har question retriever ko bhejti hai, retrieved docs ko correct docs se compare karti hai, per-question recall nikalti hai, sabka average = overall Recall@K.
- **Result & improve:** Recall = 67% aaya → low-recall questions analyze karo → fix: better embedding model, query expansion, K badhana, ya reranker.
- **Kyun programmatic?** Task objective/unambiguous hai (doc match hua ya nahi) — code se check ho jaata hai, human ki zaroorat nahi.

$$\text{Recall@K} = (\text{top-K me mile sahi docs}) \div (\text{total sahi docs})$$

Yaad Rakho Ye ID-based simple Recall@K hai. Video 13 me LLM-judge wala Contextual Recall aayega, jo chunk-IDs pe depend nahi karta.

3 Method 2 — Human-Based (Example: Chatbot Helpfulness)

Poore RAG chatbot ki **helpfulness** evaluate karni hai (application-level, quality risk). Helpfulness = accurate + complete + right tone.

- **Kyun human?** Helpfulness bahut nuanced/subjective hai — koi program automatically judge nahi kar sakta.
- **Rubric (1-5):** 5 = accurate + complete + correct tone; 3 = partially helpful; 1 = not helpful.
- **Dataset:** 50–100 diverse questions (normal, difficult, edge cases) — sirf questions, koi "correct answer" column nahi.
- **Process:** question chatbot ko → answer → human grader Q+A dekhke rubric pe 1–5 score deta hai → sabka average = helpfulness score.
- **Multiple graders:** agar 2+ graders bahut disagree karein, iska matlab rubric me ambiguity hai — rubric refine karo.
- **Advantage:** high trust. **Disadvantage:** cost — millions of users ke scale pe humans afford nahi ho sakte.

Human Evaluation ke 5 Types

#	Type	Kya hota hai
1	Direct Grading & Rating	Q+A dekhke score dena (upar wala example).
2	Red Teaming	Humans jaanbujh kar system ko attack/break karne ki koshish karte hain, breaking points dhoondhte hain.
3	A/B Testing	2 versions production me daalke real users se rating; better-rated version poora rollout.
4	Golden Dataset & Rubric Creation	Human correct answers/criteria define karta hai — ye khud ek tarah ka evaluation hai.
5	Human-in-the-Loop	Bahut complex/ambiguous cases jaha automated/LLM checks trust nahi hote, wahan judgement human ko jaata hai.

4 Method 3 — LLM-as-a-Judge (Example: UPSC Mains Auto-Grading)

Platform lakhs students ke UPSC Mains (subjective) mock-answers grade karta hai. Human graders scale pe costly; programmatic check possible nahi (long subjective text). Solution: **LLM-as-Judge** — programmatic (cheap) aur human (reliable) ke beech ka option.

- **Task/Target:** check karna ki platform ki grading human expert jaisi hai ya nahi.
- **Success criteria:** LLM ki grading, human expert ki grading ke jitna close ho.

Golden dataset — 2 steps

- **Rubric:** subject expert har question ke liye 4–5 dimensions define karta hai (concept mention hua? link explain hua? mechanism diya? examples? balanced conclusion?).
- **Human grading:** 50–100 real student answers ko expert rubric pe grade karta hai (marks + per-dimension check) — yehi golden dataset.
- **LLM ko kya diya jaata hai:** question, max marks, exact rubric, student answer. Instruction: har dimension genuinely address hua ya nahi check karo, marks do (verbosity/keyword-stuffing reward mat karo), total marks + reasoning do.
- **Iterate:** better LLM, system prompt tweak, ya rubric refine → re-run → MAE kam karte jao.

MAE = average of | LLM ke marks – Human ke marks |

Example: MAE = 2.3 → LLM average human se ± 2.3 marks deviate karta hai. Goal: MAE ko 0 ke paas lana (LLM bilkul human jaisa grade kare).

5 Kaunsa Method Kab? (Cheat-Sheet)

Method	Best for	Pros	Cons
Programmatic	Fixed/objective output (labels, doc match)	Fast, free, repeatable	Subjective text pe kaam nahi
Human	Nuanced, subjective judgement	Highest trust	Costly, slow, scale nahi
LLM-as-judge	Long unstructured text at scale	Human se sasta, scalable	Rubric/judge prompt achha hona zaroori; human se calibrate karo

6 Reference-Based vs Reference-Free Evaluation

	Reference-Based	Reference-Free
Golden dataset me	Har test case ka pre-known correct answer hota hai	Correct answer nahi; sirf rubric/standard
Judge kaise hota hai	Output ko reference se compare	Output ki quality rubric/criteria ke against
Examples	Retriever eval (correct doc IDs), UPSC eval (human marks reference)	Chatbot helpfulness (sirf questions + 1-5 rubric)

Key Insight Quick test: golden dataset me "correct answer" diya ja raha hai? Haan → Reference-Based. Nahi → Reference-Free.

Yaad Rakho Ab tak ka sab kuch mostly Offline Evaluation hai (deploy se pehle). Deploy ke baad wali Online Evaluation agla video hai.

Offline vs Online Evaluation

Deploy se pehle aur baad ki eval, signals, logging, self-improving loop

IS VIDEO ME Offline eval ke 3 benefits · Offline kaafi kyun nahi · Online eval · Signals · Logging pipeline · Self-improving loop

1 Offline Eval — Definition & 3 Key Benefits

Offline Eval = koi bhi eval pipeline jo aap deploy karne se **pehle** chalte ho. Ab tak ke saare examples (retriever eval, UPSC grader, chatbot helpfulness) offline evals the.

Benefit	Kaise kaam karta hai
1. Pre-Release Testing / Release Gate	Deploy se pehle proper testing. CI/CD ke saath automate: eval score threshold (e.g. 95%) se upar → auto-deploy; neeche → notification/rollback.
2. Version Comparison	Do options (Claude vs OpenAI model, ya alag prompts/rerankers/vector DBs) → dono pe same golden dataset + same eval; jo better score kare wahi choose.
3. Regression Testing	"Test of change" — ek cheez improve karte waqt doosri kharab to nahi hui? Golden dataset me diverse categories rakho (refund, pricing, curriculum); ek category ka sudden score drop = regression.

2 Offline Kaafi Nahi Hai — 3 Production Risks

Risk	Kya hota hai
Unanticipated Inputs	Golden dataset me sirf 200–500 anticipated questions the. Real users kuch bhi pooch sakte hain — Hindi-English mix, ambiguous half-questions, angry rants, adversarial prompt injections — jinke liye system test hi nahi hua.
Emergent / Systematic Failures	Kuch problems sirf scale pe dikhti hain: traffic spike se latency badhna, ya thousands of conversations ke baad hi dikhne wala subtle bias (e.g. non-technical users ke saath alag behavior).
Drift	Golden dataset ek fixed point-in-time pe bana tha, par real world (prices, curriculum, policies) badalta rehta hai. Dataset update na ho to eval gradually stale ho jaata hai — offline results achhe, par production me negative feedback badhta hai.

3 Online Eval — Definition

Online Eval = system ko **live production traffic** pe, deployment ke baad, real users ke interactions ke through evaluate karna.

Key Insight Sabse bada defining feature: Online eval bina answer key / golden dataset ke kaam karta hai — kyunki production me user kuch bhi naya pooch sakta hai jiska pehle se correct answer maujood nahi hota.

4 Offline vs Online — Comparison

Aspect	Offline Eval	Online Eval
Timing	Deployment se pehle	Deployment ke baad, continuously
Data	Fixed golden dataset (aap create karte ho)	Live production traffic
Answer Key	Haan — golden dataset me	Nahi — on-the-go estimate
Input	Sirf anticipated cases	Kuch bhi aa sakta hai
Catches	Regressions	Drift, surprises, emergent bugs
Best for	Release gating (CI/CD)	Drift detection, real-world health
Cost / Speed	Fast, cheap, repeatable (small dataset)	Costly at scale → sampling use hoti hai

Key Insight Offline aur Online rivals nahi, complementary hain — dono hamesha saath chalte hain, ek doosre ka replacement nahi.

5 Core Purpose — Correctness vs Normality

- **Offline** batata hai: kya app **correctly** kaam kar raha hai?
- **Online** batata hai: kya app abhi production me **normally** chal raha hai?

Example UPSC grader: Offline me humne LLM ke marks vs human ke marks ki closeness (correctness) check ki. Deploy ke baad naye student answers ka koi human reference nahi hota, to correctness measure nahi ho sakti. Instead normality: is week ke scores ka distribution pichhle weeks ke baseline se compare karo. Agar suddenly shift ho (average marks bahut high) — investigate karo, kuch abnormal hai.

Correctness ka alternative signal: **proxy signals** — thumbs-up/down rate me sudden spike, ya faithfulness jaisa metric jo correct answer jaane bina bhi check ho jaata hai (sirf context + answer compare karke).

6 Online Eval — 2 Types of Signals

Type	Matlab	Examples
Captured Signals	Pehle se present hote hain, bas store karna hai (koi computation nahi).	Thumbs up/down, latency (ms), token usage/cost
Computed Signals	Evaluator se calculate karne padte hain.	Faithfulness, answer relevance, correctness, hallucination rate, toxicity, bias/fairness

7 Online Eval Pipeline — Full Flow

Step 1 — Logging (universal first step)

Har conversation turn ko structured tarike se record karo (tool e.g. LangSmith): conversation ID, turn ID, user ID, timestamp, question, retrieved context, output, latency, tokens, cost, error codes, behavioral signals (thumbs up/down, escalation, rephrasing).

Logging property	Matlab
Non-blocking	Logging se chatbot ki latency nahi badhni chahiye.
Durable & Queryable	Future me fetch/query kar sako.

Logging property	Matlab
Late-signal attachment	Delayed signals bhi conversation ID se link ho (e.g. escalation ek din baad email se hui).
PII handling	Sensitive info (phone, card, address) store karne se pehle mask/redact karo.

Path A — Captured quantity (e.g. Latency)

Log → **Dashboard** (time-based graphs: last 1hr/24hr/1week) → **Alerting** (threshold cross hone pe Slack/email/WhatsApp) → engineer action leta hai (e.g. more compute allocate).

Path B — Computed quantity (e.g. Hallucination rate)

Log → **Sampling** (poore traffic pe evaluator chalana costly) → **Evaluator** (usually reference-free LLM-as-judge) → metric compute → Dashboard → Alerting.

Key Insight Sampling stratified honi chahiye, random nahi: conversations ko categories me baanto (thumbs-down mila, abruptly ended, escalated, money/refund-related) aur problematic categories se zyada samples lo — issues detect hone ka chance badhta hai.

8 The Self-Improving Loop

Production me jo failures pakde jaayein (galat classify, hallucination, escalation) unhe wapas **Offline Golden Dataset** me add karo → Offline Eval dobara run → improve/redeploy → naye failures → cycle continues.

Note Tool note: LangSmith jaise platforms dono handle karte hain — "Tracing" pe evaluator chalana = Online Eval; "Dataset" pe evaluator chalana = Offline Eval. Production ki koi buri conversation "Add to Dataset" se seedha offline dataset me add ho sakti hai — isi se loop close hota hai.

Model Evals — Why & What

AI Engineer ko model evals kyun chahiye, benchmarks vs custom evals

IS VIDEO ME Model evals kyun · 4-step process · Benchmarks vs Custom evals · Zomato model-selection example

1 AI Engineers Ko Model Evals Kyun Chahiye? (4 Reasons)

Model Evals ka basic purpose: LLM ki capabilities measure karna ("if you can't measure, you can't improve"). AI Engineer mostly Application Evals karta hai, phir bhi Model Evals ke 4 concrete reasons hain:

Reason	Matlab
1. Model Comparison & Selection	Naya app: OpenAI ka model use karein ya Claude? Professional setting me "koi bhi chalega" nahi bol sakte — concrete evidence chahiye.
2. Tracking Real Improvement	Naya model version aaya, manager poochta hai "upgrade karein?" — Model Evals ke numbers se justify karo ki naya sach me better hai.
3. Safety Checking	Model kitna hallucinate karta hai, kitna safe hai, jailbreak ho sakta hai ya nahi.
4. Build vs Buy	Proprietary API (Claude) use karein ya open-source model (DeepSeek) khud host karein? Cost vs capability trade-off ko compare karne me Model Evals madad karte hain.

Key Insight Model Evals ke bina aap "blind" ho — do models compare karne ka koi aur tareeka nahi hai.

2 Model Eval — Definition & 4-Step Process

Definition Model Eval ek systematic process hai jisse aap kisi LLM ki capabilities, behavior, reliability aur operational characteristics ko controlled conditions me measure karte ho.

Humans ke IQ jaisa **single number** LLMs ke liye exist nahi karta — har capability ke liye alag eval hota hai.

- 1. Decide Capability** — kaunsi capability test karni hai (reasoning, coding, safety, instruction following...).
- 2. Bring a Test** — us capability ko test karne ka mechanism/exam.
- 3. Run Under Fixed Protocol** — model "exam deta hai"; prompt, conditions sab fix, taaki repeatable ho aur sab models ko same condition mile.
- 4. Score & Interpret** — result score karke publish/interpret karna.

3 Do Types Ke Tests — Benchmarks vs Custom Evals

	Benchmarks	Custom Evals
Kya hai	Standardized, shared tests (MMLU, SWE-Bench). Sab models same test dete hain.	Aap apne actual task ka data assemble karke test banate ho.
Best for	Models ko common ground pe compare karna; universally recognized.	Aapke specific use-case ke liye kya relevant hai wo measure karna.

4 Benchmarks Akele Kaafi Kyun Nahi — Zomato Example

Task: Zomato ke incoming emails classify karna (Billing / Technical / Refund). Do model choices:

- **Model A** — leaderboard ke top pe, bahut powerful (Opus-type), par expensive: \$15 / 1M tokens.
- **Model B** — chhota, public benchmarks pe mid-table, par sasta: \$0.50 / 1M tokens.

Public benchmarks ke hisaab se Model A har capability me jeetega, to obvious lagta hai "Model A use karo". Custom eval: 200–500 past emails ka golden dataset (labelled) banake dono models run kiye:

Metric	Model A (Big/Expensive)	Model B (Small/Cheap)
Classification Accuracy	94%	91%
Urgency Accuracy	88%	87%
Cost per 1000 emails	~\$6	~\$0.21
Latency	4.1 seconds	~0.9 seconds

Key Insight Accuracy me farak bahut chhota (94% vs 91%), par cost aur latency me Model B bahut better value deta hai. Is task ke liye Model B better choice hai — jabki wo har public benchmark pe haar raha tha. Sirf benchmarks dekhte to ye conclusion kabhi nahi milta. Practical demo Video 11 me hai.

LLM Ki 8 Core Capabilities

Har benchmark inhi me se kisi ek ko target karta hai

IS VIDEO ME Knowledge & Reasoning · Coding · Math · Long Context · Vision · Agentic · Safety · Instruction Following

Ye 8 core capabilities har LLM me evaluate hoti hain. Har benchmark inme se kisi ek (ya zyada) ko target karta hai — isliye benchmarks padhne se pehle ye map clear hona zaroori hai.

1 Knowledge & Reasoning

- **Kya measure karta hai:** Model ne training me kitna factual knowledge seekha, aur kya wo multiple pieces of information connect karke reason karke sahi conclusion tak pahunch sakta hai.
- **Isme kya aata hai:**
 - Factual recall across subjects — biology, physics, chemistry, history, law, medicine, economics, philosophy.
 - Multi-step logical reasoning — kai facts ko sahi sequence me combine karke final answer tak pahunchna.
- **Labs alag se kyun evaluate karte hain:** Ye capability "model kitna intelligent hai" ka proxy maani jaati hai, aur public me sabse zyada discuss hoti hai — labs ko dikhana padta hai ki unka model widely-recognized benchmarks pe competitors ko match/beat karta hai.
- **Real-world relevance:** Research papers analyze karna, complex customer questions, technically accurate documents likhna, law/medicine/finance professionals ko assist karna.

2 Coding & Software Engineering

- **Kya measure karta hai:** Kya model actually working code likh sakta hai, aur real software engineering environments (large codebases, multiple files, dev tools) me kaam kar sakta hai.
- **Isme kya aata hai:**
 - Function-level code generation — natural-language instructions se functions likhna, edge cases handle karna.
 - Real-world bug fixing — real GitHub issue + repository diya jaata hai, model ko root cause dhoondhke correct patch banana hai.
 - Multi-file, long-horizon tasks — kai files me changes, dependencies samajhna, refactoring, naye features.
 - Command-line & sysadmin — terminal se multi-step tasks (packages install, servers configure, diagnostics).
 - API & function calling — sahi parameters ke saath calls, errors handle karna, results process karna.
- **Labs alag se kyun evaluate karte hain:** Better coding performance developer productivity badhata hai. Developers important customer group aur major revenue source hain, isliye labs yahan bahut compete karte hain.
- **Real-world relevance:** Code generation, debugging, code review, CI/CD automation, maintenance. Coding evals me leading aur weaker models ke beech sabse bada performance gap dikhta hai.

3 Mathematics

- **Kya measure karta hai:** Kya model accurate symbolic aur numerical reasoning kar sakta hai. Final answer ya correct hota hai ya incorrect — sirf convincing sound karke pass nahi ho sakte.
- **Isme kya aata hai:**
 - Grade-school & basic arithmetic — multi-step word problems, natural language ko math expressions me convert karna.
 - Competition-level (AIME jaise) — creative reasoning, combinatorics, geometry, number theory.

- Undergraduate & graduate maths — real analysis, abstract algebra, topology, probability, differential equations.
- Research-level reasoning — frontier ke paas ke problems jo experts ko bhi kai din lagate hain.
- **Labs alag se kyun evaluate karte hain:** Maths genuine reasoning ka sabse cleanest test hai — plausible-sounding par galat answer ke liye jagah nahi. Isse pata chalta hai model formal, verifiable reasoning kar raha hai ya sirf pattern-matching.
- **Real-world relevance:** Scientific computing, financial modelling, engineering simulations, data analysis — jahan precise numerical results chahiye.

4 Long Context

- **Kya measure karta hai:** Kya model bahut lambe inputs (kabhi lakhon tokens) se information effectively use kar sakta hai.
- **Isme kya aata hai:**
 - Single-fact retrieval — lambe document me chhupi ek specific information dhoondhna.
 - Multi-fact co-reference — same person/entity/constraint ko track karna jab wo kai jagah door-door mention ho.
 - Aggregation — bade text me spread information ko summarize, count, compare, pattern identify karna.
 - Code repository understanding — large codebase navigate, cross-file dependencies pe reason.
- **Labs alag se kyun evaluate karte hain:** Context-window size marketing number ban gaya hai (128K → 200K → 1M tokens), par bada window ye guarantee nahi karta ki model poori information effectively use kar sakta hai.
- **Real-world relevance:** Legal contracts review, academic papers, large codebases, lambi customer-support conversations, documents ke bade collections.

5 Vision & Multimodal

- **Kya measure karta hai:** Kya model text ke saath visual information samajh aur reason kar sakta hai.
- **Isme kya aata hai:**
 - Chart & graph understanding — axes, legends, data points, trends; exact values extract karna.
 - Document & form understanding — scanned documents, invoices, tables, forms; layout samajhna bhi zaroori.
 - GUI & screen understanding — buttons, menus, dialog boxes, input fields, app ka current state.
- **Labs alag se kyun evaluate karte hain:** Vision se models wo tasks karte hain jo text-only systems nahi kar sakte; jab bahut saare text benchmarks saturate ho chuke hain, ye differentiate karne ka important area hai.
- **Real-world relevance:** Invoices process karna, charts/research figures analyze, software interfaces navigate, medical images, engineering diagrams.

6 Agentic & Tool Use

- **Kya measure karta hai:** Kya model sirf text generate karne ke bajaye **actions** le sakta hai — browse, APIs call, files manipulate, code execute, software operate. Iske liye plan banana, sahi tools choose karna, progress track karna aur galti pe recover karna padta hai. AI evaluation ke next major frontiers me se ek.
- **Isme kya aata hai:**
 - Web browsing & persistent research — kai websites navigate, links follow, dozens/hundreds pages tak.
 - Structured tool calling & API interaction — sahi function select, valid parameters, responses handle, calls chain, errors se recover.
 - Desktop & computer use — apps ko visually operate karna (clicking, typing): word processors, spreadsheets, email, file managers.
 - Multi-turn task execution with planning — bade goal ko chhote steps me todna, intermediate results check, fail hone pe plan adjust.
- **Labs alag se kyun evaluate karte hain:** Model ke paas bahut knowledge ho sakta hai par wo use practical task me nahi laga pata. Agentic eval "model kya jaanta hai" vs "model actually kya kar sakta hai" ka gap measure

karta hai — jo frontier models me bhi abhi bada hai.

- **Real-world relevance:** Coding agents, research assistants, customer-service bots, data-entry automation, personal assistants.

7 Safety & Alignment

- **Kya measure karta hai:** Kya model responsibly behave karne ke liye trust kiya ja sakta hai — harmful requests refuse karta hai, deception/sycophancy avoid karta hai, adversarial attacks resist karta hai, user pressure me bhi principles follow karta hai.
- **Isme kya aata hai:**
 - Refusal of harmful content — cybercrime, violence, harassment, misinformation, illegal activities. Saath hi over-cautious bhi na ho (harmless requests refuse na kare).
 - Adversarial attacks & jailbreaks — prompt injection, role-playing, encoded instructions ke baad bhi safety rules maintain.
 - Truthfulness vs sycophancy — user ke belief se disagree hone pe bhi sahi answer; sirf wahi na bole jo user sunna chahta hai.
 - Cybersecurity capabilities — cryptography, reverse engineering, forensics, exploitation; potentially dangerous autonomous capabilities samajhne ke liye important.
- **Labs alag se kyun evaluate karte hain:** Safety regulatory requirement bhi hai aur major reputational concern bhi.
- **Real-world relevance:** Public ko AI deploy karne wali har organization ko safety controls test karne padte hain — legal liability, compliance, brand reputation, user trust.

8 Instruction Following

- **Kya measure karta hai:** Kya model exactly wahi karta hai jo user ne bola — na zyada, na kam (format, length, tone, style, structure, content requirements).
- **Isme kya aata hai:**
 - Verifiable constraint satisfaction — jo automatically check ho sake: word limit, bullet points, valid JSON, markdown structure, specific section headings.
 - Ambiguous/incomplete instructions — underspecified request me user ko kya chahiye samajhna, zaroorat pe clarifying question, aur reasonable defaults.
- **Labs alag se kyun evaluate karte hain:** Knowledge achha ho par model simple instructions baar-baar ignore kare to use karna frustrating hai. Ye user satisfaction se sabse directly connected capabilities me se ek hai.
- **Real-world relevance:** Almost har practical LLM use: required format me reports, style rules ke hisaab se code, structured JSON, brand-tone, coherent multi-turn conversations.

9 Quick Recap Table

#	Capability	Kya test karta hai	Real-world use
1	Knowledge & Reasoning	Factual recall + multi-step logic	Research analysis, professional Q&A
2	Coding & SWE	Working code + real repo tasks	Code gen, debugging, CI/CD
3	Mathematics	Symbolic/numerical reasoning, verifiable answers	Scientific computing, financial modelling
4	Long Context	Bade inputs (100K+ tokens) effectively use karna	Legal contracts, large codebases
5	Vision & Multimodal	Visual + text reasoning	Invoices, charts, medical images, GUI
6	Agentic & Tool Use	Actions lena (browse, API, tools)	Coding agents, research assistants
7	Safety & Alignment	Refuse harm, resist jailbreaks, truthful	Liability, compliance, brand trust

#	Capability	Kya test karta hai	Real-world use
8	Instruction Following	Exactly jo bola wahi karna	Structured outputs, brand tone

Interview One-Liner LLM evaluation 8 core capabilities ke around organized hai — Knowledge & Reasoning, Coding, Mathematics, Long Context, Vision & Multimodal, Agentic & Tool Use, Safety & Alignment, Instruction Following — aur har benchmark inme se ek ya zyada ko target karta hai.

LLM Benchmarks

Anatomy, eval harness, limitations, knowledge benchmarks ki evolution (MMLU se HLE)

IS VIDEO ME Benchmark anatomy · Eval harness · Kaun evaluate karta hai · 4 limitations · 7 knowledge benchmarks (MMLU se HLE)

1 Benchmark Kya Hota Hai?

Benchmark = ek standardized test jo LLM ki ek specific capability measure karta hai. Model evals do type ke hote hain: standardized benchmarks (fixed test, sab models pe same) aur custom evals (apni app ke hisaab se). Har benchmark ek **research paper** se aata hai (GSM8K, MMLU, SWE-Bench...) jisme dataset, run config, scoring aur aggregation define hota hai.

2 Har Benchmark Ke 4 Core Components

Example: **GSM8K** (Grade School Math, ~8K rows, 2021).

(1) Dataset / Task

- **Dataset** = Questions + Golden Answers dono. GSM8K: grade-school math word problems (e.g. "Natalia sold clips to 48 friends in April, half as many in May. Total?" → 72).
- **Task** = model ko diya jaane wala instruction: "solve this math problem and give the answer."

(2) Run Configuration

Note Ye settings dono models pe SAME honi chahiye, warna comparison invalid ho jaata hai.

Setting	Options / Matlab
Zero-shot vs Few-shot	Zero-shot: sirf question. Few-shot: pehle kuch solved examples. GSM8K classically 8-shot.
CoT vs Direct	Chain-of-Thought me model step-by-step sochta hai → accuracy badhti hai. Direct me galti ka chance zyada.
Temperature	Zero ke qareeb (deterministic). Zyada temperature = creative/random.
Max tokens	Kam rakha to CoT beech me cut ho sakta hai; zyada rakha to excess reasoning. Dono models ko same.
pass@1	Ek hi attempt, right ya wrong.
pass@k	k attempts me ek bhi sahi → correct (lenient).
maj@k	k baar poochha, jo answer sabse zyada baar aaya wahi final.
Tools	Python interpreter/web search allow karna score artificially boost karta hai. GSM8K me tools NOT allowed; SWE-Bench me (GitHub access) allowed.

(3) Scoring Method — 2 steps

- **Extraction:** model ka raw output ("The answer is 72" ya sirf "72") se sahi answer nikalna — regex ya structured output se.
- **Comparison:** Closed-ended (exact match $72 == 72$) → programmatic. Open-ended (paragraph answers) → LLM-as-a-judge ya human.

(4) Aggregation

- Har question ka score (0/1) combine karke final %: **920/1000 → 92%**.
- Complex: MMLU (57 subjects) — har subject ka alag score; questions unequal hote hain isliye **weighted mean**, simple average nahi.

3 Evaluation Process — 7-Step Loop

Model evaluation basically dataset ke upar ek loop hai. Har question ke liye:

1. **Load the item** — question + golden answer.
2. **Build the prompt** — few-shot examples inject, chat template apply, instructions add.
3. **Call the model** — pinned decoding config (temperature, max_tokens) ke saath (API model = HTTP call; local model = forward pass).
4. **Capture raw output** — verbatim log karo. Score galat lage to raw output batata hai fail model ne kiya ya parser ne.
5. **Extract the answer** — regex/parser se prose me se sahi answer.
6. **Grade** — golden key se compare → 1 (correct) ya 0.
7. **Store** — per-item result + prompt + raw output (debugging/auditing ke liye).

Loop ke baad saare per-item scores aggregate hote hain → headline number ("92% on GSM8K"). Ye loop dikhne me simple hai, par production-level me complex: error handling, retries, rate limits, batching, structured extraction.

4 Eval Harness

- Real me handle karna padta hai: "The answer is C", "C.", "Option C" — sab ko same treat karna; benchmark ka exact scoring method; hazaaron questions ko batches me bhejna; failed API requests retry; rate limits.
- **Eval harness** = ready-made system jo ye saari details handle karta hai: benchmark ke questions kahan se, kaise format, model ko kaise bhejna, answer kaise extract, sahi/galat kaise judge, final score kaise calculate.
- **Analogy:** Benchmark = exam paper; Eval Harness = poora exam administration system (instructions, answer-sheet format, checking rules, score calculation).
- **Popular tools:** lm-evaluation-harness (EleutherAI, industry standard), Inspect, HELM, DeepEval (application-eval-focused). lm-eval-harness me ek command se poora benchmark chal jaata hai.

5 Kaun Evaluation Karta Hai? (3 Stakeholders)

Stakeholder	Kyun evaluate karte hain	Unke numbers kaise padhein
Frontier Labs (OpenAI, Anthropic, Google DeepMind, Meta, Mistral, DeepSeek, Qwen)	Internally development me (pretraining/post-training decisions, kaunsa checkpoint ship karna) — invisible majority. Release gating. Marketing (launch post ki benchmark table).	Lab ke numbers ek ceiling hain — favorable conditions me measure, selection/configuration me biased.
Third-Party Evaluators (LMArena, Artificial Analysis, Epoch AI, Vals AI)	Unka business hi independent evaluation hai; neutral dikhna zaroori.	AI engineer ke liye sabse useful: har model ko khud, same conditions me run karte hain, aur cost + latency bhi measure karte hain jo labs omit karte hain.
Companies / AI Engineering Teams	Labs general capability, third parties cross-model comparability measure karte hain — par tumhare support emails sahi price pe classify hote hain ya nahi, ye tumhe khud measure karna padta hai.	Custom evals (Video 11).

Interview Tip Sirf frontier lab ke published numbers pe blindly trust mat karo — poocho "evaluation kisne kiya, kis condition me?"

6 Benchmarks Ki Limitations (Interview-Important)

Limitation	Kya hota hai	Solution / Note
Contamination	Public benchmark ke questions+answers internet pe hote hain; model ki pre-training me chale jaate hain → model reason nahi kar raha, memorize kar chuka.	Private ya dynamic benchmarks (regularly update).
Saturation	Naya benchmark aata hai → sab models 25-35%. Time ke saath sab 90%+ pe cluster → models ko differentiate nahi kar pata → retire.	GSM8K, MMLU, old SWE-Bench saturated maane jaate hain.
Configuration Gaming	Labs apne model ko favorable conditions (extra tools) dete hain, competitor ko default → 5-10% artificial farak.	Config ki poori transparency; nahi di to number trust mat karo.
Aggregation Gaming	Multi-subject benchmark me sirf average dikhta hai, subject-wise weaknesses chhup jaati hain (Physics achhi, Economics kharab par overall 85%).	Domain-specific app ke liye subject-wise breakdown maango.

7 Knowledge Capability — 7 Famous Benchmarks Ki Evolution Story

Knowledge = training me LLM ne kitna world knowledge retain kiya (weights/biases me parametric knowledge). Ye sabse fundamental capability hai; reasoning, coding jaisi capabilities baad me "emergent behavior" ki tarah aayi. Har naya benchmark kisi **problem** ko solve karne aaya — ye story yaad rakho:

Benchmark	Year	Role in the story	Status (2026)
MMLU	2020	"Mother of all benchmarks" — Breadth	Saturated
TruthfulQA	2021	MMLU ki branch: Reliability/Truthfulness	Saturated → SimpleQA
AGIEval	2023	Branch: existing human exams reuse	Saturated
GPQA	2023	Branch: Depth (breadth ki jagah)	Near saturation
MMLU-Pro	2024	Branch: MMLU ki flaws fix	Near saturation
SimpleQA	2024	Reliability branch v2 (TruthfulQA ki jagah)	Active
HLE	2025	Sab converge: Breadth × Depth	Active

(1) MMLU — Massive Multitask Language Understanding (Sept 2020) · Saturated

- **Tests:** sirf **breadth** of knowledge (depth nahi).
- **Dataset:** 57 subjects, ~14,000 MCQs (4 options), GRE/USMLE/AP jaise real exams aur textbooks se.
- **Scoring:** (a) model directly answer generate kare, ya (b) har option ki log-probability — sabse high wala answer. Dono me 2-3% farak.
- **Config:** 5-shot, direct (no CoT), temperature 0, no tools. Prompt format bahut sensitive — thoda wording change se score wave karta hai (config gaming risk).
- **History:** GPT-3 (2020) 43.9% vs human expert baseline 90%. GPT-4 (2023) 86%. 2024 tak frontier 86-92% pe cluster → saturated.
- **Kyun saturate hua:** ~6.5% questions galat/ambiguous → koi 100% nahi kar sakta, sab ek jagah cluster.
- **Measure NAHI karta:** reasoning depth, calibration, open-ended generation, multilingual (English-only, Western curriculum-centric).
- **Known issues:** label errors (6.5%), high contamination (2020 se public), prompt-format gaming.

(2) TruthfulQA (Sept 2021) · Saturated — Reliability branch

- **Problem:** MMLU breadth test karta hai, par truthfulness guarantee nahi. Internet pe misconceptions bhi hain ("knuckle cracking se arthritis") — bade models zyada misconceptions bhi absorb kar sakte hain.
- **Famous finding:** bade/zyada capable models is benchmark pe zyada fail karte the — "capability ≠ alignment" idea yahin se popular hua.
- **Dataset:** 817 adversarial questions, 38 categories (common human misconceptions).
- **Scoring:** Generation (model khud answer generate kare), MC1 (ek sahi answer maan ke log-prob compare), MC2 (multiple sahi answers, probabilities add) — MC2 default/standard.
- **Quirk:** "Zero-shot" bola jaata hai par har baar 6 fixed unrelated example questions prompt me jaate hain (static hone se zero-shot maana).
- **Measure nahi karta:** factual recall (MCQ hai), honesty under pressure (ye MASK benchmark check karta hai).
- **Saturation:** 2024 tak — RLHF/instruction-tuning se models zyada truthful. Contamination training stage pe nahi, **alignment stage** pe hui (dataset alignment fine-tuning data ka part ban gaya).
- **Replaced by:** SimpleQA aur MASK.

(3) AGIEval (April 2023) · Saturated — Human-exams reuse branch

- **Philosophy:** naya benchmark banane ke bajaye existing human exams (SAT, LSAT, Gaokao) LLM ko do. Fayde: naya dataset banane ki mehnat nahi; real human baseline milta hai (estimated nahi, measured).
- **Dataset:** 20 exam sections, 8000+ questions, bilingual (English + Chinese — pehla bilingual benchmark). 18/20 MCQ, 2 short-answer.
- **Results:** GPT-4 launch pe 58%, human average 67%, human toppers 91% → LLMs top humans se peeche.
- **Saturation:** 2024-25 tak frontier models topper baseline (91%) ke qareeb → ab use nahi hota.

Interview Tip Jab news aaye "LLM ne IIT-JEE me humans ko beat kar diya" — iska matlab ye NAHI ki model human intelligence se aage nikal gaya. Ek exam crack karna ≠ multi-step reasoning, tool use, long-horizon tasks. Ye sirf exam-specific knowledge test karta hai.

(4) GPQA — Google-Proof Q&A (Nov 2023) · Near saturation — Depth branch

- **Philosophy:** MMLU breadth test karta hai par questions easy the. GPQA **depth:** sirf 3 subjects (Physics, Chemistry, Biology), par PhD-level research-grade.
- **"Google-proof":** non-expert ko Google access + 30 minutes de do, tab bhi solve nahi kar payega. Har question 2 domain experts ne validate kiya.
- **3 subsets:** Main (448), Extended (546), **Diamond (198)** — sabse difficult; "GPQA score X%" = generally Diamond.
- **History:** GPT-4 (2023) 39% → GPT-4o (2024) 56% → OpenAI o1 (reasoning) 78%. PhD experts 69.7% (paper me claim 81.3% tha — discrepancy, isliye "PhD ko beat kiya" marketing pe pura trust mat karo). Grok 4 (2025) ~87%.
- **Measure nahi karta:** general graduate knowledge (sirf 3 sciences), open-ended problem solving, reasoning trace correctness (galat reasoning se bhi sahi answer pe marks mil jaate hain).
- **Issues:** bahut kam questions (198) → statistical confidence kam; PhD baseline disputed; contamination badh raha (2-3 saal purana); sirf 3 domains.
- **Status (2026):** near saturation — reasoning models 80%+.

(5) MMLU-Pro (2024) · Near saturation — "MMLU rebuilt to fix its flaws"

- **Philosophy:** MMLU achha product tha, sirf flaws fix karo — naya banane ke bajaye upgrade.
- **3 changes:** (1) 4 options → **10 options** (elimination-based guessing mushkil); (2) trivia/noisy questions hataye, **reasoning-based** questions add; (3) 57 subjects → **14 broad categories** (better balance).
- **Proof:** reasoning-capable models non-reasoning se ~20 points aage — ab sochna padta hai.
- **Fun fact:** "MMLU-Redux" benchmark nahi, ek paper hai jisne prove kiya MMLU me 6-8% questions incorrect hain — isi ne MMLU-Pro ko trigger kiya.
- **Dataset:** 12,000 questions, 14 disciplines.

- **Measure nahi karta:** open-ended generation, reasoning trace correctness, calibration. Reasoning models ko unfair advantage (reasoning-heavy design).
- **Status (2026):** near saturation — models 80-90%.

(6) SimpleQA (2024, OpenAI) · Active — Reliability branch v2

- **Special:** abhi saturate nahi hua — in 7 me sabse "alive".
- **Format innovation:** MCQ NAHI — short-answer, fact-seeking questions; options nahi, khud answer generate karna. Difficulty automatically badhti hai.
- **Dataset:** 4326 short factual questions — jin par us waqt ka state-of-the-art (GPT-4) fail hua tha.
- **Calibration bhi measure:** Correct / Incorrect / **Not Attempted** ("I don't know"). Not-attempted ka matlab model apni knowledge limits jaanta hai (hallucinate nahi kar raha).
- **3 metrics:** Correct (overall), Correct-given-attempted (sirf attempt kiye hue me accuracy), F-score (dono ka harmonic mean).
- **Result:** jo model MMLU pe 88% tha, SimpleQA pe ~40%.
- **Measure nahi karta:** long-form factuality, everyday factual reliability (dataset rare/hard questions se bana).
- **Issues:** LLM-as-judge grading (judge improve hota rehta hai → 2-saal-purane results compare nahi ho sakte); answer staleness ("current world #1 rugby player"); GPT-4 ke against selection bias.

(7) HLE — Humanity's Last Exam (Jan 2025) · Active — Breadth × Depth

- **Core idea:** MMLU se breadth + GPQA se depth, dono combine karke ek massive benchmark.
- **Naam ki thesis:** agar models is (closed-ended, broad, expert-level, unambiguous) benchmark ko crack kar lein, to closed-ended Q&A test karne ko kuch nahi bachega — evaluation ko open-ended, agentic tasks ki taraf jaana padega.
- **Scale:** 2500 questions, 100+ subjects (classics se rocket engineering), 1000+ experts, 500+ institutions, 50 countries — massive collaborative effort.
- **Format:** 80% short-answer, 20% MCQ; 10% multimodal (image-based) — kisi benchmark me pehli baar visual data.
- **Anti-contamination:** public set (2500, released) + private set (institute ke paas, kabhi public nahi) — naye models private set pe bhi test ho sakte hain.
- **Calibration:** answer ke saath confidence % maangte hain (e.g. "80% confident"), phir RMS calibration error.
- **History:** Jan 2025 me models single-digit %; Grok 4 → 24%, GPT-5 → 25%, Gemini → 38% (2025); Gemini 3 Pro 38%+ (2026 tak).
- **Measure nahi karta:** open-ended/agentic problem solving, everyday usefulness, vision (sirf 10%), multilingual (English-only).
- **Issues:** kuch questions disputed (3000 → 2500); LLM-judge grading errors; failure-filter selection bias (2024 ke frontier models ki failures se bana).
- **Status:** abhi knowledge + reasoning + math combined ke liye state-of-the-art benchmark.

Quick recap — status snapshot

Benchmark	Tests	Format	Status
MMLU	Breadth of knowledge	4-option MCQ, 57 subjects	Saturated
TruthfulQA	Reliability/Truthfulness	MC1/MC2/Generation	Saturated
AGIEval	Human-exam comparison	MCQ + short-answer	Saturated
GPQA	Depth (Phy/Chem/Bio)	4-option MCQ	Near saturation
MMLU-Pro	Breadth + light reasoning	10-option MCQ, 14 disciplines	Near saturation
SimpleQA	Factuality + Calibration	Short-answer (no options)	Active
HLE	Breadth × Depth + Calibration	80% short-answer, 20% MCQ, 10% multimodal	Active

Interview One-Liner Benchmark ek standardized test hai 4 parts ke saath — dataset, run config, scoring method, aggregation. Par contamination, saturation, config gaming aur aggregation gaming ki wajah se published numbers pe blindly trust mat karo: hamesha check karo kisne evaluate kiya, kis condition me, aur subject-level breakdown hai ya nahi.

Interview One-Liner Knowledge benchmarks MMLU (breadth) se shuru hue → reliability (TruthfulQA → SimpleQA), human-exam comparison (AGIEval), depth (GPQA), aur MMLU repair (MMLU-Pro) me branch hue → sab HLE me converge hue jo breadth + depth + calibration test karta hai, aur jiske saturate hone ke baad evaluation open-ended agentic tasks ki taraf jaayegi.

LLM Leaderboards

4 types, kyun blindly trust nahi, AI Engineer ki tarah kaise padhein

IS VIDEO ME Leaderboard kya · 4 reasons · 5 stakeholders · 4 types · 7 trust issues · 5-step reading framework

1 LLM Leaderboards Kya Hain?

Leaderboard = public ranking/comparison table jo dikhata hai ki different LLMs ek common set of evaluations pe kaisa perform karte hain. Benchmark ek exam hai; uska result kahin publish/display karna hi leaderboard hai.

Leaderboards kyun exist karte hain (4 reasons)

1. **Common reference se compare** — sabko same exam, kaun top kaun last turant pata. Trust bhi: OpenAI khud bole "hum best" to self-interest lagta hai; neutral third party bole to credible.
2. **Model selection guide** jab khud evals run karne ke resources na ho — 100s models test karna costly hai.
3. **Saturation detect karna** — top models ek jagah cluster (92-94%) karein to benchmark saturate ho raha hai.
4. **Naye models discover karna** — top 3-4 hamesha same (OpenAI, Google, Anthropic), par thoda neeche scroll karo to naye/cheaper models milte hain.

2 Kaun Use Karta Hai? (5 Stakeholders)

Stakeholder	Kaise use karte hain
AI Engineers	100 candidates ko 5 tak shortlist; task-specific leaderboard (math app ke liye math leaderboard).
Frontier Labs	Competitive positioning + release gating. Naya model competitor ke purane model ko bhi beat na kare to release nahi. (Google ka "Nano Banana" stealth mode me leaderboard pe aaya, achha perform kiya tab officially launch hua.)
Researchers	Nayi technique genuinely help kar rahi hai ya nahi — shared baseline ke against.
Policymakers & Safety Institutes	Capability trends track — forecasting/regulation ke liye.
Open-weight ecosystem	Discovery ke liye sabse underrated: 3-log ki chhoti lab bhi frontier model ke saath same table pe dikh sakti hai.

3 4 Types Ke Leaderboards

Type	Kaise kaam karta hai	Example	Limitation
Benchmark-specific	Ek single benchmark ke results pe ranking. "Is benchmark pe best kaun?"	HLE ka apna leaderboard, sirf MMLU/HumanEval/GSM8K/GPQA	Bahut narrow view; overall model kaisa hai pata nahi.
Multi-benchmark (sabse useful)	Kai benchmarks combine karke overall score + practical factors (cost/token, latency, speed, context window, price-to-performance).	LiveBench (23 objective tasks, 7 categories, contamination-free); Artificial Analysis (intelligence/speed/cost boards)	Composite weights subjective ho sakte hain.

Type	Kaise kaam karta hai	Example	Limitation
Human-preference	Benchmark score nahi, human votes: 2 anonymous models (A vs B) me se user better chunta hai. Helpfulness, clarity, writing, creativity, tone measure.	LMarena (Chatbot Arena) "battle mode"	Human bias — log lambe, confident, formatted, "agreeable" answers prefer karte hain; voters tumhare users jaise na hon.
Application-specific	Ek real-world domain/task ke around (bug fixing, tool calling, SQL, medical, agents, long docs).	Berkeley Function-Calling Leaderboard (BFCL)	Sirf us domain ke liye relevant.

4 Leaderboards Ko Blindly Trust Kyun Nahi Kar Sakte (7 Reasons)

#	Reason	Detail
1	Benchmark performance real apps me transfer nahi hoti	Benchmark questions clean/self-contained; real world me ambiguous requests, missing info, company-specific data, multi-turn, tool failures. (Kaggle jeetna ≠ achha real-world data scientist.)
2	Contamination score inflate karti hai	High score = genuine capability, memorization, ya similar-questions se familiarity — kuch bhi ho sakta hai.
3	Leaderboard overfitting / Goodhart's Law	Companies model ko leaderboard ke liye tune karti hain (LMarena pe jeetne ko lambe, confident, human-pleasing answers). Goodhart: jab measure target ban jaata hai, wo achha measure nahi rehta. (Car company sirf mileage pe focus kare to overall quality gir jaayegi.)
4	Overall ranking subjective design choices pe	Kaunse benchmarks include, scores kaise normalize, kis capability ko kitna weight — inhe badalne se ranking badal sakti hai; aur ye transparent nahi hota.
5	Chhote score differences meaningful nahi	84.3 vs 84.1 — 0.2 ka farak 3rd aur 5th rank bana sakta hai; noise ho sakta hai. Confidence intervals na ho to adjacent ranks ko tied samjho.
6	Human-preference me human bias	"Better lagne wala" answer zaroori nahi "sabse sahi" ho.
7	Scores stale/incomplete/self-reported	Purane versions, naye models missing, discontinued models, ya company ka khud submit kiya (unverified) result.

5 AI Engineer Ki Tarah Leaderboards Kaise Padhein — 5 Steps

Key Insight "Leaderboards are a filtering tool, not a decision tool." Leaderboard se seedha ek model select nahi karte — sirf top candidates shortlist karte hain.

Step 1 — Apni requirements pehle likho

Leaderboard kholne se pehle: task type, latency budget, cost ceiling, context needs, deployment constraints (on-prem? open weights? data retention?). Isse rank #1 ki taraf blindly bias nahi hoge.

Step 2 — Sahi board choose karo (famous wala nahi, task se match karne wala)

Task	Board
Agents	SWE-bench Pro / Terminal-Bench / OSWorld
Tool use	BFCL
Chat	Matching LMarena sub-arena
RAG	MTEB (embedding models ki ranking)
Budget-constrained	Price + throughput dikhane wala board (jaise Artificial Analysis)

Step 3 — Board ko sahi se padho (7 checks)

- Exactly kya score ho raha hai? (answer key, test suite, human vote, ya LLM judge?)
- Model kisne run kiya? (independent evaluator ya vendor self-report?)
- Kis inference compute pe? (reasoning-effort settings score aur bill dono badalti hain)
- Eval set kitna purana? (static + public = contamination risk)
- Saturated hai? (Top 10 agar 2-3 points ke andar hain to ho chuka)
- Confidence intervals kahan hain? (na ho to adjacent ranks tied)
- Composite hai to weights kya hain? (unpublished weights = unpublished opinion)

Step 4 — 3-5 candidates tak cut karo

Hard filters pehle: license, price ceiling, latency, apne region me availability.

Step 5 — Apna khud ka eval chalao (yahi asli decision)

Apne real traffic se 50-few hundred examples lo — ye kisi bhi public benchmark se better hota hai. Quality, cost, latency aur failure behaviour pe score karo.

Interview One-Liner Leaderboards filtering tool hain, decision tool nahi — 100 models ko apne task ke leaderboard type se 3-5 candidates tak narrow karo, phir final call se pehle apne real traffic pe khud ka evaluation chalao.

Custom Model Evals — Text-to-SQL Case Study

Requirements, shortlisting, custom eval (Cricinfo example)

IS VIDEO ME 3-stage funnel · Requirements (cost, latency, correctness) · Leaderboard shortlisting · Custom eval pipeline · Live results

1 Ye Session Kahan Fit Hota Hai

Yaad Rakho Model Evals = Benchmarks (generic, standardized) + Custom Model Evals (apna data/task). Benchmarks/leaderboards sirf top candidates dete hain ("filtering tool") — final decision ke liye apna custom eval chalana padta hai (Video 7 aur 10). Ye video usi ka practical demo hai.

2 Case Study — ESPN Cricinfo "Ask Cricinfo"

- **Problem:** Live match ke dauraan users bahut questions poochte hain ("Virat Kohli ka Pakistan ke against batting average?"). Pehle human analyst team query likhke commentary team ko deti thi — bade matches (India vs Pakistan) me 5 min me 500+ questions pe scale nahi hota tha.
- **Solution:** "Ask Cricinfo" — user natural language me poochta hai → system use **SQL query** me convert karta hai → database pe run → result seedha user ko. Human analyst ki zaroorat khatam, infinitely scalable.
- **Humara role:** hum Cricinfo ke AI engineers hain, Text-to-SQL app banani hai. Sabse pehla sawal: **kaunsa LLM?** Random guess se nahi ("Fable use kar lete hain, abhi ka top model hai" companies me nahi chalta) — proper evaluation process chahiye.

Note Ye RAG nahi hai — model ko external documents nahi diye ja rahe. Sirf database ka schema diya jaata hai (jo hamesha same rehta hai), usi ke basis pe SQL generate karna hai.

3 Model Selection — 3-Stage Flow

Stage	Kya karna hai
1. Requirements gather karo	Task, cost, latency, context, deployment, correctness — sab clearly likh lo.
2. Leaderboard shortlisting	Requirements ke basis pe filter karke 5-10 candidate models. (Leaderboards = filtering tool, selection tool nahi.)
3. Custom eval	In 5-10 models pe apna eval chalo; jo sabse best score kare wahi final.

4 Stage 1 — Requirements Gathering

(a) Task

Model ka kaam: question + database schema dekhke **ek SQL query** generate karna. LLM ko jaane wala structure: system instruction ("tum text-to-SQL generator ho, SQLite syntax me sirf ek query return karo, koi explanation nahi") + database schema + user ka question.

- **matches table:** match details (teams, ground, date, umpire...).

- **deliveries table:** ball-by-ball data (batsman, bowler, runs, wicket...).
- Domain sirf **IPL** tak narrow rakha gaya (scope simple karne ke liye).

(b) Cost Ceiling

Business manager se budget pucha: is case study me **₹3 lakh/month** (illustrative; class exercise me ₹5 lakh tha).

Business angle: retention play — user jitna zyada website pe rukega, utne ads, utna revenue. Isliye feature ka ROI justify karna padta hai; cost ceiling isi trade-off se aata hai.

Cost calculation — step by step:

1. **Per-query tokens:** Input (system prompt + schema + question) ≈ 400 ; Output (SQL) $\approx 100 \rightarrow$ ratio 4:1.
2. **Daily volume:** business judgement se ~ 5000 questions/day (traffic spiky hoga, par average nikalna hai).
3. **Formula:** neeche.

$$\text{Per-query cost} = (\text{Input price} \times \text{Input tokens} \div 1,000,000) + (\text{Output price} \times \text{Output tokens} \div 1,000,000)$$

$$\text{Daily cost} = \text{Per-query cost} \times \text{Daily questions}$$

$$\text{Monthly (USD)} = \text{Daily cost} \times 30 \quad \text{Monthly (INR)} = \text{USD} \times 95$$

Worked example — Claude Fable 5: Input \$10/M, Output \$50/M $\rightarrow$ per-query = $(10 \times 400/1M) + (50 \times 100/1M) = \$0.004 + \$0.005 = \mathbf{\$0.009/query}$. Lecture me final result **₹12.82 lakh/month** bataya gaya — budget ka $\sim 4x$, isliye Fable reject. Cheaper tier (jaise Claude Sonnet, $\sim 1/4$ th price) dekhna padega.

Source Note Lecture ka figure (\$450/day $\rightarrow$ ₹12.82 lakh) in inputs se match nahi karta: $\$0.009 \times 5000 = \$45/\text{day} \rightarrow \$1,350/\text{month} \rightarrow \approx ₹1.28 \text{ lakh}$ (10x kam). Source notes me bhi yahan calculation me confusion tha. Formula yaad rakho, exact number pe mat ruko.

Key Insight Cost calculation ek core AI engineering skill hai — sirf model capability nahi, budget-fit bhi utna hi important hai.

(c) Prompt Caching — cost optimization

- Agar prompt ka bada portion (system prompt + schema) har query me same hai aur sirf question badalta hai, to provider **prompt caching** deta hai — cached portion ka baar-baar poora price nahi dena padta.
- **Cache miss (pehli query):** thoda zyada charge (normal \$10/M ki jagah \$12.5/M) — cache processing ka extra cost.
- **Cache hit (refresh window ke andar):** bahut kam (sirf $\sim \$1/M$).
- **Refresh cycles:** 5-minute ya 1-hour cache. High traffic (Cricinfo — har 5 min me questions) $\rightarrow$ 5-minute; low traffic (~ 1000 q/day) $\rightarrow$ 1-hour.
- **Mechanism:** KV caching (attention ke Key-Value vectors cache hote hain). Impact: cost $\sim 90\%$ tak kam (₹12 lakh $\rightarrow$ ₹6-7 lakh potentially).
- **Limitation:** RAG chatbots me utna kaam nahi karta — retrieved context har query me badalta hai, koi fixed repetitive portion nahi.

(d) Latency · (e) Context window · (f) Deployment · (g) Correctness

- **Latency:** users live match ke dauraan curiosity se poochte hain, turant answer chahiye. Target **2-3 seconds** (5 sec bhi zyada). Heavy reasoning models (lamba sochne wale) suitable nahi. (Research assistant hoti to latency matter na karti.)
- **Context window:** bada matter nahi karta — har question independent hai, koi multi-turn/session memory nahi. (Chatbot ya coding assistant me bahut matter karta.)

- **Deployment:** koi sensitive/private data nahi, isliye public APIs (Anthropic, OpenAI...) open hain; public APIs ki reliability generally on-prem se better.
- **Correctness:** critical. Cricket fans records ke finicky hote hain — ek galat answer ("sabse zyada runs Bumrah ne banaye") ka screenshot viral ho jaayega. Hallucination allowed nahi; coding leaderboards (SQL = coding task) aur custom eval dono se verify karna hai.

Requirement	Decision
Task	Text-to-SQL application
Cost ceiling	≤ ₹3 lakh/month
Latency	2–3 seconds max
Context window	Major factor nahi
Deployment	Public API preferred (better reliability)
Correctness	Extremely important — no hallucination

5 Stage 2 — Leaderboard Shortlisting (5-10 Candidates)

Text-to-SQL leaderboards kyun reject hue

- **BIRD-SQL:** outdated (purane models, kuch fine-tuned models jo directly use nahi ho sakte). Spider aur **LiveSQLBench** bhi dekhe; LiveSQLBench outdated tha (purane models listed) aur methodology unclear.
- **Fallback:** SQL generation ek **coding activity** hai — isliye general coding leaderboard ko proxy/alias treat karo.
- **Chosen:** LLMStats.com (third-party, kai leaderboards aggregate karke overall coding rating; 146 models, latest models updated, cost + speed + context info).

Shortlisting process

Step 1 — Cost filter. Leaderboard blended price deta hai (input:output ratio 4:1 pe based). Cricinfo ka apna ratio bhi exactly 4:1 (400:100) tha, isliye ye number seedha use hua.

$$\text{Blended price} = (4 \times \text{Input price} + 1 \times \text{Output price}) \div 5$$

$$\text{Monthly cost} = (\text{Tokens per query} \times \text{Blended price} \div 1,000,000) \times \text{Daily questions} \times 30 \times 95$$

146 me se jo ₹3 lakh se upar the reject → ~50–60 models bache.

Step 2 — Rank karo (2 criteria): coding rating aur latency/speed (characters per second). Dono ko 0–1 me **min-max normalize** kiya:

$$\text{Normalized} = (\text{Value} - \text{Min}) \div (\text{Max} - \text{Min})$$

$$\text{Final score} = 0.9 \times \text{Norm. Rating} + 0.1 \times \text{Norm. Latency}$$

Note Latency ko sirf 10% weight kyun? Output sirf ~100 tokens ki chhoti SQL query hai — model thoda slow ho to farak nahi padta. Output bahut bada (pura essay) hota to latency ka weight zyada hota.

Top-10 me cost-effective options bhi the (open-source Chinese models — MiniMax M3, Qwen, GLM) jo GPT/Claude se score me bahut peeche nahi the par fraction of cost pe.

Custom eval ke liye final 5 models

Model	Note
GPT 5.6 Tera	Leaderboard top candidate — expensive par strong.

Model	Note
Kimi K3	Moonshot AI, 2.7 trillion params, open-weights; bahut hype (Fable ke qareeb, 1/3rd cost).
Grok 4.5	xAI ka model.
Claude Sonnet 5	Anthropic family ka sole representative (Tera se cheaper).
MiniMax M3	Cheap Chinese open-source model.

Note GLM/Qwen bhi consider ho sakte the (time constraint se ek hi Chinese model). Meta ka Llama Spark region restriction (sirf US) ki wajah se exclude hua.

6 Stage 3 — Custom Eval Pipeline (Steps)

Step	Kya kiya	Detail
1. Dataset	IPL data Kaggle se	2008–2024 ka dataset; simplification ke liye sirf 2020–2024. 2 CSV files: matches, deliveries.
2. Database	CSV → SQLite	Sirf 2 tables, 2020–24 data.
3. Schema extract	Schema ek baar extract karke file me store	Table/column names/types system prompt me hamesha include hote hain, taaki LLM ko database structure pata ho.
4. Golden dataset	20 question + verified-correct SQL pairs	Neeche detail.
5. Sanity check	1 question + 1 test model	Candidate list se bahar ka model (GPT-4o) — bug to nahi setup me.
6. Multi-provider API	OpenRouter	5 models alag providers ke, sabki API alag. OpenRouter = ek single API se kisi bhi provider ka model; LangChain integration; free credits.
7. Candidate list	5 tuples	Har tuple: model naam + OpenRouter slug.
8. Evaluator	Sabse important step	Neeche detail.
9. Orchestration	Main pipeline	Schema + golden dataset load → DB connect → har model ke liye har question bhejo → generated SQL lo → golden aur generated dono DB pe run → evaluator compare → result log → sab models ke liye repeat.

Golden dataset (Step 4) — details

- Golden dataset = 20–50 (yahan 20) manually-curated question + verified-correct SQL pairs — evaluation ka "answer key".
- Ideally human data-analyst banata hai (question sochta hai, SQL likhta hai, DB pe run karke validate karta hai). Demo me LLM se banwaya ("bahut difficult questions") aur DB pe run karke syntax errors check kiye.
- Representative** ho actual user distribution ka — easy/medium/hard ka mix (e.g. 50 me 10 easy, 20 medium, 20 hard). **Variety** ho: joins, sub-queries, window functions.
- Har entry: question, golden SQL, expected row count, **order-sensitivity flag** (ORDER BY chahiye ya nahi).
- Size trade-off:** human effort/cost + zyada questions = zyada API calls = zyada eval cost. Typical 50–500. Production me jahan model fail ho wahan se naye questions add karte jao.

Evaluator (Step 8) — result tables compare karo, SQL text nahi

Key Insight String-matching galat approach hai: ek hi result ke liye multiple different SQL queries likhi ja sakti hain (text alag, functionally same). Isliye queries ka text nahi, unke **output tables** compare karte hain.

- Row count check** — alag hai to seedha mismatch.
- Value normalization** — 2.0 aur 2 same; 2.999 aur 2.99 jaise near-equal floats same.
- Row-by-row structuring** — dono tables list-of-tuples format me.

4. **Sorting-based comparison** — ORDER BY nahi hai to dono ko sort karke compare (row order vary ho sakta hai). Order matter karta hai to bina sort compare — verify ho ki LLM ne sahi ORDER BY use kiya.
5. Sab match → correct; warna incorrect. (Kuch chhote edge-case checks bhi the.)

7 Live Results — Kya Nikla

Model	Accuracy	Notes
GPT 5.6 Tera	~80% (4 errors / 20)	Strong par expensive.
Kimi K3	~50-55%	Bahut hype, par poor real-task performance + slow (reasoning model, latency issue); SQL syntax errors bhi.
Grok 4.5	~90%	Sabse fast, sabse accurate, sabse sasta bhi (~₹2.5 lakh/month, Tera se lagbhag aadha).
Claude Sonnet 5	~85%	Bahut fast, reliable, cost ~₹2.84 lakh/month.
MiniMax M3	~65%	Sabse cheap, par SQL syntax errors (Chinese open-source models me zyada dekhe gaye).

Key practical lessons

1. **Leaderboard hype ≠ real-task performance.** Kimi K3 news me chhaya hua tha, par custom eval me poor (slow + inaccurate).
2. **Normalization vs denormalization:** analytical/reporting workloads ke liye denormalized tables prefer hote hain (kam joins → kam error-prone SQL); transactional (OLTP) ke liye normalization. Cricinfo jaisi company ka production analytics data likely already denormalized hoga.
3. Is test me Chinese open-source models (MiniMax, Kimi) me SQL syntax errors zyada; GPT, Grok, Claude consistently valid syntax.
4. **Statistical confidence:** 20 questions ke dataset me ek galti = 5% swing. Production-grade eval ke liye bada dataset, aur/ya same eval multiple baar run karke average (har API call independent hai, results vary karte hain).

Final decision

Do finalists: **Grok 4.5** (fastest, cheapest, ~90%) vs **Claude Sonnet 5** (~85%, thoda costlier, par Anthropic ki API reliability/infrastructure zyada trusted). Sirf numbers me Grok aage, par vendor reliability/trust bhi valid factor hai — final call = business judgement + engineering confidence. Consensus na bane to voting bhi valid mechanism hai.

Note High-stakes application (medical) me extra price dekar zyada correct model choose karte hain; consumer-facing low-stakes me thoda kam accurate par sasta chalega. Decision hamesha application-context-dependent hota hai.

Interview One-Liner Model selection ek three-stage funnel hai — pehle concrete requirements likho (task, token economics se nikala cost ceiling, latency, context, deployment, correctness bar), phir leaderboards ko sirf filtering tool banake 5-10 candidates shortlist karo, aur finally apne data pe custom eval (golden dataset + result-set comparison evaluator, string-matching nahi) chalao — kyunki leaderboard hype task-specific performance guarantee nahi karta.